Atelier de professionnalisation

Partie 5.3 : Mise en production

Application mobile de gestion des notes de frais — GSB

Introduction

Votre application gsb-frais est maintenant complète : connexion, affichage, ajout, modification et suppression des notes de frais. Mais elle ne vit encore que sur votre ordinateur, dans Expo Go ou dans l'émulateur. Dans cet atelier, vous allez la transformer en véritable application Android : un fichier APK, installable sur un téléphone sans Expo Go et sans ordinateur allumé à côté, que vous pourrez partager avec un simple lien.

À l'issue de l'atelier, vous saurez :

- configurer un projet Expo pour un build de production (identifiant, versions, communication sécurisée avec l'API) ;
- construire un APK avec EAS Build, sur les serveurs d'Expo, sans Android Studio ;
- installer, tester et diagnostiquer l'application sur un vrai téléphone ;
- partager l'APK en connaissant les risques de sécurité associés (signature, lien public, intégrité).

Prérequis

- Les parties 1 à 4 terminées : l'application fonctionne en CRUD complet, et l'URL de l'API est centralisée dans config.js (adresse https://apigsbfrais.wasmer.app/api/).
- Node.js installé et le projet qui se lance avec npx expo start.
- Un téléphone Android (ou, à défaut, l'émulateur Android Studio).
- Une adresse e-mail pour créer un compte Expo gratuit.

Comprendre : développement et production

Jusqu'ici, vous avez lancé l'application avec npx expo start : c'est un fonctionnement de développement. Expo Go charge votre code JavaScript depuis le serveur de développement (Metro) lancé sur votre ordinateur, avec rechargement à chaud et messages d'erreur détaillés. Si Metro s'arrête, l'application ne fonctionne plus.

Un build de production est un exécutable autonome : le code de l'application, optimisé, est embarqué dans le fichier. Sur Android, ce fichier existe en deux formats :

Format	Usage	Installable directement ?
APK (.apk)	Distribution directe : transfert de fichier, lien de téléchargement, test interne.	Oui
AAB (.aab)	Format exigé par le Play Store, qui génère ensuite les APK adaptés à chaque téléphone.	Non

Dans cet atelier, vous allez produire un APK. La compilation se fait avec EAS Build (Expo Application Services) : le projet est envoyé aux serveurs d'Expo, qui le compilent et le signent, puis vous renvoient le fichier.

L'application n'a plus besoin du serveur de développement (Metro) ni de l'ordinateur : le code JavaScript est empaqueté dans l'APK et s'exécute seul. Il est optimisé : minifié, compilé en bytecode (Hermes), sans les outils de développement (rechargement à chaud, messages d'erreur détaillés, mode développement désactivé), donc plus rapide et plus compact.

Étape 1 : Préparer le projet

1. Vérifier l'état du projet

Dans un terminal, à la racine du projet, vérifiez que les dépendances sont cohérentes avec votre version d'Expo. Corrigez les problèmes signalés avant de continuer : un projet qui a des avertissements en développement produit rarement un build sain.

```
$ npx expo-doctor
$ npx expo install --check
```

2. Compléter app.json

Le fichier app.json décrit l'identité de votre application. Ouvrez-le et vérifiez que les champs name, slug et version sont renseignés. Ajoutez surtout android.package, l'identifiant unique de l'application. L'identifiant peut être vu un peu comme le nom de domaine pour une application web.

```
{
  "expo": {
    "name": "GSB Frais",
    "slug": "gsb-frais",
    "version": "1.0.0",
    "android": {
      // ToDo : ajouter la propriété "package" avec un identifiant unique,
      // par exemple "com.votrenom.gsbfrais" (supprimer ce commentaire : le JSON
n'en accepte pas)
    }
  }
}
```

À vous de réfléchir / chercher

1. À quoi sert android.package ? Que se passerait-il si deux applications installées sur le même téléphone avaient le même identifiant ?
2. Pourquoi ne peut-on pas utiliser un identifiant déjà pris par une autre application publiée sur le Play Store ?

⚠ Un changement natif exige un nouveau build

Modifier android.package ou toute autre configuration native dans app.json touche l'application compilée. Si vous le corrigez après un premier build, il faudra reconstruire l'APK : un simple rechargement du code JavaScript ne suffit pas.

3. Mettre à jour votre suivi de version git

EAS Build envoie votre projet aux serveurs d'Expo : il s'appuie sur le fichier .gitignore pour savoir quels fichiers ne pas envoyer (node_modules, par exemple).

Retirer le fichier config.js du .gitignore, l'ajouter à l'index (git add) et faire un commit. Sans lui, l'application compilée ne pourrait pas joindre l'API.

Étape 2 : Vérifier la communication avec l'API

Votre application appelle l'API à l'adresse https://apigsbfrais.wasmer.app/api/ : la communication est chiffrée. C'est important, car depuis Android 9 une application installée refuse par défaut le trafic « en clair » (http://), alors qu'en développement tout passe.

1. Contrôler config.js

Ouvrez config.js et vérifiez que l'adresse est bien celle de l'API hébergée, en https:// et terminée par une barre oblique :

```
export const API_URL = "https://apigsbfrais.wasmer.app/api/";
```

2. Tester l'API depuis le navigateur

Ouvrez https://apigsbfrais.wasmer.app/api/ dans le navigateur du téléphone, sur le Wi-Fi puis en 4G. Observez le cadenas dans la barre d'adresse : la connexion est sécurisée, et le serveur répond depuis n'importe quel réseau.

> **À vous de réfléchir / chercher**
>
> 1. Pourquoi Android bloque-t-il par défaut le trafic en http ? Qu'est-ce qu'une personne connectée au même Wi-Fi que vous pourrait lire ?
> 2. Imaginez que l'API soit en http://. Votre application afficherait « Network request failed » une fois installée. On pourrait contourner le blocage avec le plugin expo-build-properties et la propriété usesCleartextTraffic. Pourquoi serait-ce une mauvaise solution pour une application qui envoie un mot de passe et un jeton ?

Étape 3 : Configurer EAS Build

1. Créer un compte et se connecter

Créez un compte gratuit sur expo.dev. Indiquez le nom de l'app que vous avez mis dans app.json. Puis n'exécutez pas les commandes indiquées, cliquez directement sur Go to dashboard.

Installez ensuite l'outil en ligne de commande EAS CLI et connectez-vous :

```
$ npm install --global eas-cli
$ eas login
$ eas whoami
```

La dernière commande affiche le nom du compte connecté : c'est votre vérification.

2. Générer eas.json

```
$ eas build:configure
```

Cette commande crée à la racine du projet le fichier eas.json, qui décrit des profils de build. Ouvrez-le et repérez la section build.

√ Would you like to automatically create an EAS project for @alphanie/gsb-frais-app? ... yes

√ Which platforms would you like to configure for EAS Build?» All

3. Modifier eas.json

Ouvrez le fichier eas.json. La propriété build définit différent profils de build. Le profil preview sert à produire une version de test installable directement. Complétez-le pour qu'il génère un APK :

```
{
  "build": {
    "preview": {
      "distribution": "internal",
      "android": {
        // ToDo : demander explicitement un fichier apk (propriété buildType)
      }
    },
  }
}
```

Écrire ce qu'on veut obtenir

La distribution « internal » produit déjà un APK par défaut. Écrire buildType explicitement rend l'intention lisible pour la personne qui reprendra le projet, et protège contre un changement de comportement par défaut.

À vous de réfléchir / chercher

1. Pourquoi le profil production, laissé vide, ne donne-t-il pas un fichier que vous pouvez installer directement sur votre téléphone ?
2. Que signifie la distribution « internal » ? À qui est-elle destinée, par opposition à une publication sur le Play Store ?

Étape 4 : Lancer le build

1. Construire l'APK

Exécuter la commande suivante pour lancer le build :

```
$ eas build --platform android --profile preview
```

Répondez comme suit :

√ Generate a new Android Keystore? ... yes

Puis attendez…

⚠ Le build peut prendre du temps

Le build se fait sur les serveurs d'Expo. Avec un compte gratuit, il est placé dans une file d'attente à basse priorité : comptez de quelques minutes à plus d'une demi-heure, selon l'affluence. Le terminal affiche un lien vers la page de suivi ; vous pouvez aussi retrouver vos builds avec eas build:list ou sur expo.dev. Profitez-en pour répondre aux questions de cette étape.

2. Comprendre la signature

Un APK est toujours signé avec une clé privée, stockée ici par EAS sur les serveurs d'Expo. La commande eas credentials permet de consulter cette clé et d'en télécharger une copie.

À vous de réfléchir / chercher

1. À quoi sert la signature d'une application Android ? Que vérifie le téléphone lors d'une mise à jour ?
2. Que se passe-t-il si vous perdez la clé de signature ? Et si elle tombe entre de mauvaises mains ?

Étape 5 : Installer et tester sur un téléphone

1. Installer l'APK

Depuis le navigateur de votre téléphone, connectez vous à expo.dev, sélectionnez en haut à droite le nom de votre équipe ou votre nom personnel en fonction de ce que vous avez choisi lors de votre build.

Cliquez sur le nom de votre app, puis dans le menu déroulant choisissez Builds, cliquez sur votre build, puis cliquez sur Install. Téléchargez le fichier .apk et ouvrez-le.

Vous devrez peut-être autoriser l'installation d'applications inconnues sur votre téléphone. Autorisez-le temporairement puis désactivez l'autorisation.

Android affichera un avertissement, car l'application n'a pas été examinée par le Play Store : vous devrez autoriser l'installation depuis cette source.

Si vous n'avez pas de téléphone Android, vous pouvez installer le build sur l'émulateur avec la commande eas build:run -p android.

2. Parcourir les scénarios de test

Testez l'application comme un utilisateur et cochez chaque scénario réussi :

Scénario à tester	Résultat attendu	OK
Ouvrir l'application	L'écran d'accueil s'affiche, sans Expo Go ni ordinateur	
Se connecter avec un compte valide	Arrivée directe sur Dashboard	
Consulter la liste des notes de frais	Les notes s'affichent depuis l'API	
Ajouter, modifier puis supprimer une note	Chaque opération est visible dans la liste	
Fermer complètement l'application puis la rouvrir	Session conservée grâce au stockage dans AsyncStorage	
Se déconnecter	Retour à l'écran d'accueil	
Activer le mode avion puis tenter une action	Un message d'erreur s'affiche. Option : modifiez le message d'erreur pour indiquer à l'utilisateur de vérifier sa connexion internet.	

3. Diagnostiquer un échec

Si l'application affiche « Network request failed » ou ne charge aucune donnée, parcourez dans l'ordre cette liste :

- Le téléphone a-t-il bien accès à Internet (Wi-Fi ou 4G, mode avion désactivé) ?
- L'adresse de config.js commence-t-elle par https:// et se termine-t-elle par une barre oblique ?
- Ouvrez l'adresse de l'API dans le navigateur du téléphone : obtenez-vous une réponse, avec un cadenas ?
- Le fichier config.js figurait-il dans .gitignore au moment du build ?

À vous de réfléchir / chercher

1. Activez le mode avion, puis ouvrez l'application et essayez de vous connecter. Que constatez-vous ? Que doit faire une bonne application dans ce cas, et la vôtre le fait-elle ?
2. Supposons que vous venez de modifier app.json (android.package) : pourquoi faut-il reconstruire l'APK, alors qu'en développement une modification d'un écran apparaît immédiatement ?

Étape 6 : Partager l'APK

1. Obtenir le lien de partage

En cliquant sur Install sur la page du build, sur expo.dev, vous obtenez un QR code et un lien pour installer l'application. Par défaut, ce lien est accessible à toute personne qui le possède, sans compte Expo. Vous pouvez aussi télécharger le fichier .apk et le transmettre autrement.

2. Calculer l'empreinte du fichier

Pour permettre à un destinataire de vérifier que le fichier reçu n'a pas été modifié en route, calculez son empreinte SHA-256 depuis l'ordinateur et communiquez-la séparément du fichier (commande Windows):

```
> certutil -hashfile gsb-frais.apk SHA256
```

À vous de réfléchir / chercher

1. Quel risque présente un lien de téléchargement accessible à toute personne qui le possède ? Comment peut-on restreindre l'accès aux membres du projet ? (Consultez la documentation sur la distribution interne.)
2. En quoi l'empreinte protège-t-elle le destinataire ? Pourquoi faut-il la transmettre par un autre canal que le fichier lui-même ?
3. Un camarade installe votre APK chez lui, en 4G. Qu'est-ce qui fonctionnera ? Que change le fait que l'API soit accessible depuis Internet pour la sécurité des données ?

Remise du travail

Déposez dans la zone de dépôt:

- le lien de la page de build de votre APK ;
- l'empreinte SHA-256 du fichier ;
- une capture d'écran de l'application installée sur le téléphone (ou l'émulateur) affichant la liste des notes de frais ;
- votre fichier app.json et votre fichier eas.json ;
- vos réponses écrites aux questions.

Pour aller plus loin

> **Pistes de réflexion**
>
> 1. HTTPS : l'API est en HTTPS. Que protège-t-il exactement, et que ne protège-t-il pas (pensez à ce qui est stocké sur le téléphone, au jeton et à ce que contient l'APK) ?
> 2. Variables d'environnement : au lieu d'écrire l'adresse https://apigsbfrais.wasmer.app/api/ en dur dans config.js, comment utiliser une variable propre à chaque profil de build (développement, préproduction, production) ? Renseignez-vous sur les variables EXPO_PUBLIC_ et sur la propriété env des profils de eas.json.
> 3. Mises à jour : EAS Update permet de corriger le code JavaScript d'une application déjà installée sans reconstruire l'APK. Quelles modifications pourront être livrées ainsi, et lesquelles imposeront un nouveau build (pensez à l'étape 1) ?
> 4. Publication : que faudrait-il changer pour publier sur le Play Store (format du fichier, compte, signature) ?

Ressources

- ↗ EAS Build — mise en route
- ↗ Construire un APK avec EAS Build
- ↗ Distribution interne — Expo
- ↗ Versions de l'application (version, versionCode)
- ↗ Android — Configuration de sécurité réseau
- ↗ EAS Update — mises à jour OTA